

Исключения и обработка ошибок

Цель

Закрепить навыки работы с ошибками в Python

Задание 1. Обработка деления на ноль

Напишите программу, которая принимает два числа от пользователя и выводит результат их деления. Используйте обработку исключений, чтобы корректно обработать ситуацию, когда пользователь вводит 0 в качестве второго числа.

```
In [1]: try:
        a = float(input("Введите первое число: "))
        b = float(input("Введите второе число: "))

        result = a / b
        print(f"Результат деления: {result}")

    except ZeroDivisionError:
        print("Ошибка: деление на ноль невозможно!")
```

Ошибка: деление на ноль невозможно!

Задание 2. Обработка некорректного ввода

Расширьте предыдущую программу, чтобы она также обрабатывала ситуацию, когда пользователь вводит строку вместо числа. Используйте несколько блоков except для обработки разных типов исключений.

```
In [2]: try:
        a = float(input("Введите первое число: "))
        b = float(input("Введите второе число: "))

        result = a / b
        print(f"Результат деления: {result}")

    except ZeroDivisionError:
        print("Ошибка: деление на ноль невозможно!")

    except ValueError:
        print("Ошибка: введено не число. Попробуйте снова.")

    except Exception as e:
        print(f"Непредвиденная ошибка: {e}")
```

Ошибка: введено не число. Попробуйте снова.

Задание 3. Создание собственных исключений

Напишите программу, которая вычисляет сумму списка целых чисел. Создайте свои собственные классы исключений для обработки ситуаций, когда в списке есть хотя бы одно чётное или отрицательное число. Используйте оператор `raise` для генерации исключений.

```
In [3]: class EvenNumberError(Exception):
        def __init__(self, value):
            super().__init__(f"Ошибка: найдено чётное число ({value})")

class NegativeNumberError(Exception):
    def __init__(self, value):
        super().__init__(f"Ошибка: найдено отрицательное число ({value})")

# Основная программа
numbers = [1, 3, 5, -7, 9] # пример списка

try:
    total = 0
    for num in numbers:
        if num < 0:
            raise NegativeNumberError(num)
        if num % 2 == 0:
            raise EvenNumberError(num)
        total += num

    print(f"Сумма списка: {total}")

except NegativeNumberError as e:
    print(e)

except EvenNumberError as e:
    print(e)
```

Ошибка: найдено отрицательное число (-7)

Задание 4. Обработка ошибок индексации

Напишите программу, которая принимает от пользователя индекс элемента списка и выводит значение этого элемента. Используйте обработку исключений для корректной обработки ситуаций, когда пользователь вводит индекс, выходящий за пределы списка.

```
In [6]: fruits = ["яблоко", "банан", "апельсин", "груша"]

try:
    index = int(input("Введите индекс элемента списка: "))
    value = fruits[index]
    print(f"Элемент с индексом {index}: {value}")

except IndexError:
    print("Ошибка: индекс выходит за пределы списка!")
```

```
except ValueError:
    print("Ошибка: индекс должен быть целым числом.")
```

Ошибка: индекс выходит за пределы списка!

Задание 5. Обработка ошибок преобразования типов

Напишите программу, которая принимает от пользователя строку и преобразует её в число с плавающей точкой. Используйте обработку исключений для корректной обработки ситуаций, когда пользователь вводит строку, которую невозможно преобразовать в число.

```
In [7]: try:
        user_input = input("Введите число: ")
        number = float(user_input)
        print(f"Вы ввели число: {number}")
    except ValueError:
        print("Ошибка: введённая строка не может быть преобразована в число.")
```

Ошибка: введённая строка не может быть преобразована в число.

Задание 6. Обработка ошибок импорта модулей

Напишите программу, которая импортирует модуль `math` и использует функцию `sqrt()` для вычисления квадратного корня числа, введённого пользователем. Используйте обработку исключений для корректной обработки ситуаций, когда модуль `math` не может быть импортирован или функция `sqrt()` не может быть вызвана для отрицательного числа.

```
In [8]: try:
        import math
    except ImportError:
        print("Ошибка: не удалось импортировать модуль math.")
    else:
        try:
            num = float(input("Введите число для вычисления квадратного корня: "))
            result = math.sqrt(num)
            print(f"Квадратный корень числа {num} равен {result}")
        except ValueError:
            print("Ошибка: нельзя вычислить квадратный корень из отрицательного числа.")
```

Ошибка: нельзя вычислить квадратный корень из отрицательного числа.